

NYC Urban Mobility Data Explorer

Technical Report | Group 10 | ALU Software Engineering Full-Stack Development Summative Assessment

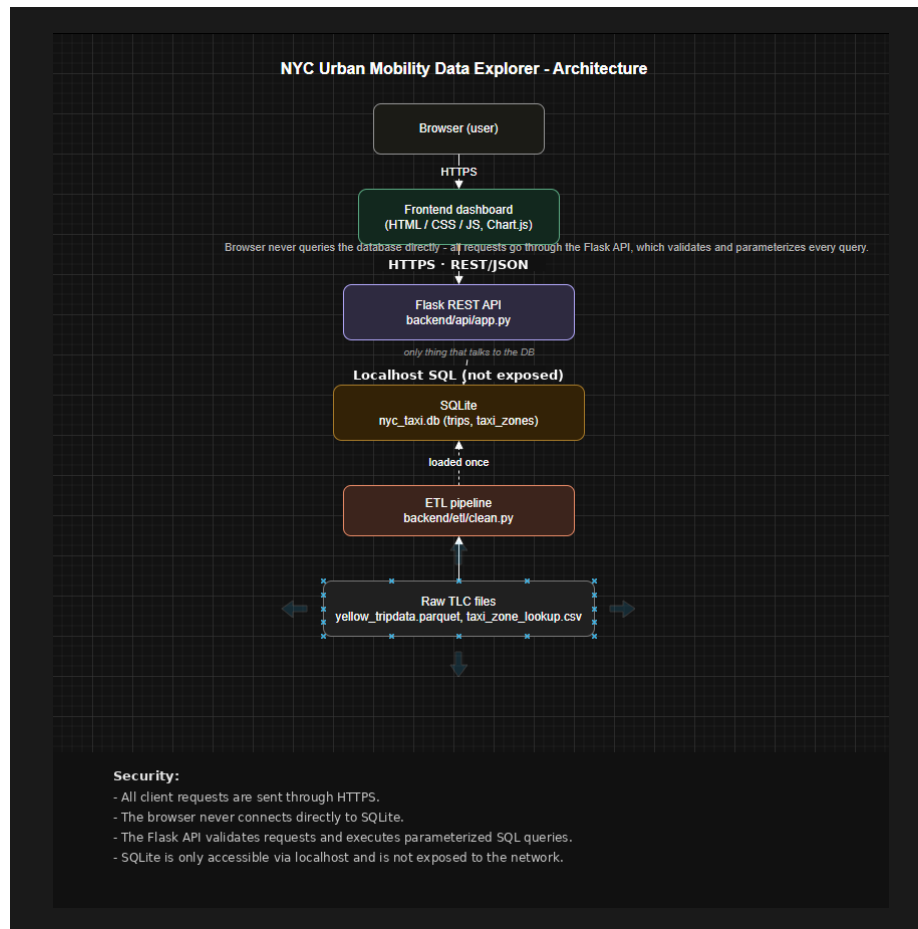
Gabriel Mugisha · Olivier Dusabamahoro · James Kanneh

Problem

Raw NYC TLC taxi data comes as disconnected files — trip records, a zone lookup table, and zone boundaries — with no way to answer basic questions like which boroughs earn the most, when demand peaks, or how airport trips differ from local ones. This project cleans and loads a full month of trip data (7.67 million raw rows) into a relational database, serves it through a REST API, and exposes it through a dashboard that lets a user explore those patterns without writing SQL.

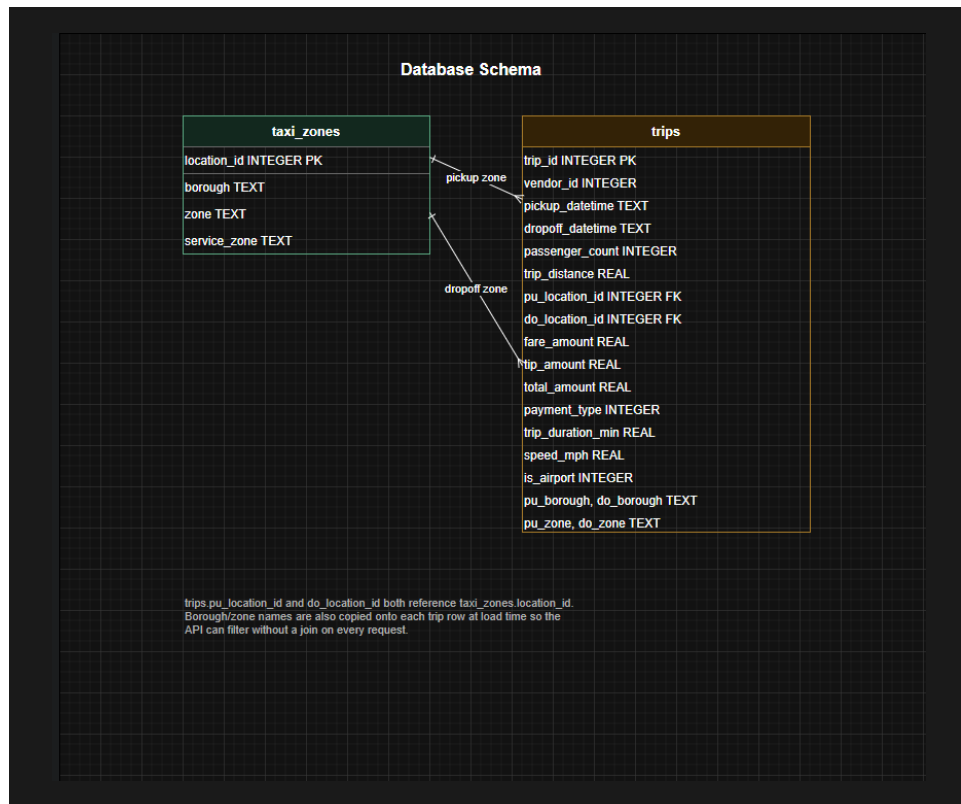
Architecture

The browser only ever talks to the static frontend and to the Flask API over HTTPS. The Flask API is the only component allowed to touch the database — it validates every request and uses parameterized SQL over a localhost-only connection, so the database is never exposed to the network directly. The ETL pipeline is a separate offline job: it runs once to load and clean the raw data into SQLite before the API or frontend ever serve a request. Keeping these three concerns (data loading, data serving, data display) separate let the three of us build them in parallel. Communication between the client and server occurs over HTTPS using REST/JSON. The SQLite database is never directly accessible to clients; all database operations are performed through the Flask API using parameterized SQL queries over a localhost-only connection.



Database Design

The schema is one denormalized trips table plus a taxi_zones reference table, joined once at load time rather than on every query. Borough and zone names are copied onto each trip row during the ETL so the API can filter and group by location without a runtime join. Five indexes (pickup_date, pu_borough, hour, payment_type, is_airport) cover the dashboard's actual filter and group-by patterns rather than indexing every column.



Algorithms

Three algorithms were implemented by hand in backend/etl/algorithms.py instead of using Python's built-in sort, heapq, or Counter, and are exposed live through /api/dsa-benchmark:

Algorithm	Used for	Complexity
QuickSort	Sorting trips or zones by a numeric key	$O(n \log n)$ avg, $O(n^2)$ worst
Min-heap	Finding the top-10 busiest pickup zones without sorting all ~260 zones	$O(n \log k)$
HashMap (chaining)	Counting trips per borough	$O(n)$ average

Insights

97.6% of the 7.67 million raw rows passed cleaning (7,485,632 kept). The rest were rejected for zero passengers, zero distance, negative fares, bad timestamps, or out-of-range dates — each logged with its reason in dead_letter.csv rather than silently dropped.

- **Manhattan dominates volume, not fare** — 91% of trips but the lowest average fare (\$10.56), since most rides are short (2.22 mi avg).
- **Outer boroughs and airports cost far more** — Staten Island (\$43.09) and Queens (\$35.59) avg fares dwarf Manhattan's; airport trips average \$39.60 vs \$10.29 for local trips.

- **Traffic is genuinely slow** — 91% of trips never exceed 20 mph.
- **Credit card dominates and is the only tipped method** — 72% of trips, \$2.53 avg tip, vs \$0.00 for cash (TLC doesn't capture cash tips).
- **Demand follows residents, not transit** — the busiest pickup zones (Upper East Side, Midtown) are where people live and work, not where they arrive.

Reflection

Being able to define the schema for the trips table and the structure of stats.json upfront enabled us to work in parallel, as Olivier worked on the ETL process and the database, Gabriel on the API, and James on the dashboard, without stepping on each other's toes. One of the most important insights we had was that determining when a record is bad should be done by inspecting the data and not by making assumptions up front; rather than silently ignoring bad records, logging all of them allowed us to validate our cleaning process afterwards. If we had more time, extending the time series from one to twelve months would have made sense, justifying switching from SQLite to PostgreSQL.